DATABASE

PostgreSQL,

the source of truth.

Tables, rows, SQL, relationships, constraints, and the habits that keep app data safe.

tables SQL keys joins psql

PERSISTENCE

Apps need data that survives.

NOT ENOUGH

- ✕ Memory disappears on restart.
- ✕ Files are awkward for concurrent writes.
- ✕ Searching and filtering gets slow.
- ✕ Relationships become manual work.

DATABASE GIVES US

- ✓ Durable storage.
- ✓ Safe concurrent access.
- ✓ Fast querying.
- ✓ Data integrity with constraints.
- ✓ Transactions for all-or-nothing work.

the database is where important app state lives.

WHY RELATIONAL?

Tables plus rules plus relationships.

PostgreSQL stores data in tables and lets us define how records relate to one another.

INTEGRITY

Constraints stop invalid data before it enters.

QUERIES

SQL can filter, sort, aggregate, and join data.

TRANSACTIONS

Multiple changes can succeed or fail together.

CORE CONCEPTS

Database words have exact meanings.

Term	Meaning
Database	A named collection of tables.
Table	Structured storage for one type of data.
Row	One record in a table.
Column	One field every row can have.
Schema	The definition: names, types, constraints.
Primary key	Unique identifier for each row, usually id.
Foreign key	A column that points at another table's primary key.

SCHEMA SHAPE

Types tell PostgreSQL what belongs.

SERIAL

Auto-incrementing integer for ids.

INTEGER / FLOAT

Whole numbers and decimals.

VARCHAR / TEXT

Short bounded text or long text.

BOOLEAN

True or false.

TIMESTAMP

Date and time.

DATE

Date only.

DDL

A table is storage with constraints.

```
CREATE TABLE users (
  id         SERIAL PRIMARY KEY,
  username   VARCHAR(50) NOT NULL UNIQUE,
  email      VARCHAR(100) NOT NULL UNIQUE,
  bio        TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);
```

CONSTRAINTS

- ✓ PRIMARY KEY uniquely identifies rows.
- ✓ NOT NULL requires a value.
- ✓ UNIQUE prevents duplicates.
- ✓ DEFAULT fills a value automatically.

INSERT / SELECT

Write rows.
Read rows back.

```sql
INSERT INTO users (username, email, bio)
VALUES ('ada_lovelace', 'ada@example.com', 'First programmer');
```

```sql
SELECT * FROM users;

SELECT username, email
FROM users
WHERE id = 3
ORDER BY created_at DESC
LIMIT 10;
```

UPDATE / DELETE

The most dangerous missing word is WHERE.

CORRECT

```sql
UPDATE users
SET bio = 'Mathematician and writer'
WHERE id = 1;

DELETE FROM users
WHERE id = 4;
```

CATASTROPHE

```sql
UPDATE users
SET bio = 'oops';

DELETE FROM users;
```

without WHERE, every row is invited to the party.

FOREIGN KEYS

Tables can point at other tables.

```sql
CREATE TABLE posts (
  id      SERIAL PRIMARY KEY,
  title   VARCHAR(200) NOT NULL,
  body    TEXT,
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE
);
```

`user_id` must match an existing `users.id`. Cascade means deleting the user deletes their posts too.

JOIN

JOIN puts related data back together.

```
SELECT posts.title, users.username
FROM posts
JOIN users
  ON posts.user_id = users.id;
```

JOIN TYPES

- ✓ JOIN returns rows with matches in both tables.
- ✓ LEFT JOIN keeps all rows from the left table.
- ✓ Unmatched right-side columns become NULL.

PSQL

The CLI is your database microscope.

```
psql -U postgres
psql -U postgres -d mydb

\l       -- list databases
\c mydb  -- connect
\dt      -- list tables
\d users -- describe table
\q       -- quit
```

SEED FILES

A `.sql` file can drop, create, and insert development data.

```
psql -U postgres -d mydb -f seed.sql
```

PRODUCTION HABITS

Databases reward careful engineers.

BEGINNER GOTCHAS

- × Missing `WHERE` on update/delete.
- × Confusing `NULL` with empty string.
- × Using double quotes for string values.
- × Assuming SQL casing changes meaning.

INTERNSHIP CONTEXT

- ✓ Migrations change schemas safely.
- ✓ Indexes speed frequent lookups.
- ✓ Transactions protect multi-step work.
- ✓ Parameterized queries prevent SQL injection.
- ✓ GUI tools help inspect data visually.